

Efficient random number generation using an entropy store

Calum Grant

Abstract—We present new algorithms for generating random integers from entropy input, that are extremely efficient from an entropy perspective. The method introduces an *entropy store* that is available to cache unused entropy between invocations, and makes rejection sampling much more efficient. This allows us to bypass theoretical limits of discrete distribution generating trees, and converges to asymptotic efficiency much faster than traditional batching approaches. For example we can shuffle a deck of 52 cards with an entropy efficiency of about 0.99999992 using a 32-bit entropy store, compared with classical algorithms that only have an efficiency of about 0.81. The algorithms are simple and efficient to implement, and have practical applications when generating variates directly from hardware-based entropy which is often a performance bottleneck.

Index Terms—Bernoulli distribution, discrete random variable, entropy conversion, entropy efficiency, entropy store, random number generation, uniform distribution, weighted distribution.

I. INTRODUCTION

In this paper, we will study generating perfectly distributed integers from an entropy source. A typical example of this is using fair coin flips to roll a fair die or to perform a perfect shuffle of a deck of cards, but random numbers are ubiquitous in other areas such as security, AI, and simulation.

The general problem is one of *entropy conversion*, where entropy in one form X needs to be converted to entropy in a different form $f(X)$, where X is a discrete random X having Shannon entropy $H(X)$ [31]. A fundamental property is that you cannot get more entropy out than you put in, i.e. $H(X) \geq H(f(X))$. [4] We will be mainly focussed on improving the *efficiency* of entropy conversion, defined as $\eta = \frac{H(f(X))}{H(X)}$.

Whilst this problem has been studied extensively, there are theoretical limits on efficiency because

algorithms must always fetch entropy in quantised units, and any excess entropy is unused. An optimal algorithm for generating uniform variables from coin flips must fetch up to 2 extra bits of entropy per output. [19]

To mitigate these entropy losses, it is possible to generate random variables in batches. The major drawback with batching schemes is that they do not scale very well and have limits on their capacity and efficiency.

In this paper we will explore a fundamentally different approach, by allowing entropy conversion algorithms access to an *entropy store* (ES) in the form of a large uniformly distributed integer variable, and allow the algorithm to put any left over entropy back into the store. By first looking at the available operations on a uniform variable, we can compose them in such a way as to minimise the entropy loss to far below optimal algorithms like Knuth-Yao [19].

The entropy efficiency depends only on the ratio of the size of the store to the size of the number generated, and an exact calculation is given in Theorem 1. For n bits of stored entropy, the entropy loss is $\Theta(n2^{-n})$ and the efficiency is $1 - \Theta(n2^{-n})$, which exceeds traditional batching techniques which have efficiency $1 - \Theta(n^{-1})$ [20].

For example to roll a 6-sided die with a 32-bit entropy store has an entropy efficiency of > 0.99999997 . To shuffle a deck of 52 cards with a 32-bit entropy store has an entropy efficiency > 0.99999992 .

A. Contribution

We present new algorithms, called *entropy store algorithms*, that have lower setup costs and higher

amortised entropy efficiency than classical algorithms or batched algorithms. Table I summarises the algorithm, where n is the number of bits in an integer, k is the size of the weighted distribution W .

Output	Uniform	Bernoulli	Weighted
Input	Unbiased coin or dice		
Precision	Exact		
Entropy in	$H(W) + O(n2^{-n})$ (amortised)		
Setup time	$O(n)$		$O(kn)$
Space	$O(n)$		$O(kn)$
Per output	$O(n \log n)$		$O(n \log n)$

TABLE I
ENTROPY STORE ALGORITHM CHARACTERISTICS.

We show that this algorithm is faster other random number generators when reading directly from a hardware entropy source, where the limiting factor is the rate of entropy input.

B. Related work

Entropy conversion is a well studied subject starting when Von Neumann [23] proposed the first known algorithm for generating perfectly uniform variables (a "dice roll") from an unbiased coin (a "coin flip") - shortly after Claude Shannon's initial work on entropy [31]. Von Neumann's rejection sampling algorithm works by fetching the smallest power of 2 greater or equal to the number being generated. If the number is in range, return it, otherwise retry. This algorithm is elegant but not optimal in its entropy efficiency.

Knuth and Yao [19] devised an optimal algorithm, using a discrete distribution generating tree based on an inverse Huffman encoding [17] of each output, and proved that no other binary decision tree could on average use fewer decisions.

The drawbacks with the original Knuth-Yao algorithm are the setup and space costs: the decision trees are explicitly constructed can get quite large. [29, 28] Lumbruso [22] devised an optimal dice rolling algorithm, that is no more efficient than Knuth-Yao, but requires no setup and is simple to implement. Bacher *et al.* [2] analyse the Von Neumann and Knuth-Yao algorithms from an entropy perspective, using Lumbruso's implementation, with a particular focus on generating unbiased

permutations from unbiased coin flips. There are always up to 2 bits of entropy loss (or "toll") for each uniform variable generated, depending on the number being generated. Only exact powers of 2 can be generated without entropy loss. [19]

Lemire [21] compares different practical uniform generators, and describes a uniform generator that is efficient from a CPU perspective, but is very inefficient from an entropy perspective, by avoiding CPU division operations. Lemire's use case is for entropy that comes from a pseudorandom source that can be quickly generated.

Various methods have been explored for generating arbitrary weighted distributions. The Knuth-Yao algorithm can be used to generate arbitrary distributions from binary entropy.

The *alias method* devised by Walker [35] and optimized by Vose [34] maps a uniform variable to an output, and then uses a second Bernoulli variable to look up an aliased output. This is $O(n)$ in setup time, and $O(1)$ in output time, but is not optimal in terms of entropy efficiency.

Han and Hoshi [13] devised an optimal interval algorithm to convert biased or unbiased coin-flips to an arbitrary distribution, with an overhead of up to 3 bits of entropy per output. The algorithm works by dividing the $[0, 1)$ real interval according to the output distribution, and stops at bit n when the fetched binary number falls entirely within the output interval in the n^{th} bit. Han and Hoshi show that this algorithm has an optimal entropy consumption. Wanatabe [36] analyses this algorithm using an information spectrum, and Oohama [24, 25] analyses the performance this algorithm. Han and Hoshi's algorithm can also be used to read biased inputs and form part of a Markov chain.

Saad *et al.* [30] introduce Fast Loaded Dice Roller (FLDR) algorithm and Draper and Saad [7] the Amplified Loaded Dice Roller (ALDR) algorithm to generate more efficient discrete distribution generating trees (DDG). They observe that DDG-based algorithms like Knuth-Yao can be very large and slow, so propose a table-based approach with optimizations, which creates a faster random distribution generator within 2 bits per sample of the output entropy for ALDR, and 6 bits per output for

FLDR.

Entropy extraction studies how to convert the entropy *from* a variety of different distributions. When the input distribution is unknown, Von Neumann [23] gives an algorithm to extract unbiased entropy from a biased coin, which is again elegant but suboptimal in its entropy efficiency. Peres [27] devised a way to recursively use the previously-discarded outputs to provide an algorithm with perfect amortised entropy efficiency, and the method was generalised by Pae [26] to extract entropy from arbitrary but identically distributed unknown distributions (“weighted M-dice”) with perfect amortised efficiency. Interestingly, it appears to be easier to extract entropy than to generate it.

Universal hashing can be used to turn unknown distributions into fair bits. Vembu *et al.* [33] analyse the maximum rate at which entropy can be extracted. Goldreich’s Leftover hash lemma [11] calculates the length of possible output. Trevisan [32] implements a universal hash extractor for unknown distributions with a known entropy.

A common proposal to reach asymptotic efficiency for random number generation is to use batching to generate multiple outputs at the same time. [2, 5, 13, 19, 20, 22] Batching spreads the entropy loss over the batch size, but does not recycle entropy beyond the batch, so is still constrained by the Knuth-Yao limit. However batching schemes suffer from increased size and complexity, and are less flexible because you must plan beforehand which numbers to generate. Batching does not always scale well because it requires arbitrary floating point precision or data structure sizes, particularly for the less optimised algorithms. Batching for random permutations is still an open problem. [2]

Kozen and Soloviev [20] show that the efficiency of batching k outputs is $1 - \Theta(k^{-1})$ in the general case, as well as developing a coalgebra and methods for combining randomness conversions, leveraging the efficiency gains of batching.

Fill and Huber [9, 16], describe the *randomness recycler* protocol which is a general purpose strategy to identify excess entropy in an algorithm and store it for the next state. This idea has been used to create entropy-optimal uniform [22, 15] or weighted

[15] variables, but entropy is not recycled between invocations, so their method does not improve on the efficiency established by Knuth-Yao and Han-Hoshi.

II. ALGORITHMS FOR ENTROPY CONVERSION

In this section we’ll define some basic building blocks (Algorithm 1-4), and then use these to create an efficient generator for uniform variables (Algorithm 5).

We’ll then build on Algorithm 4 to create a generator for Bernoulli variables (Algorithm 6) and weighted variables (Algorithm 7). All of these algorithms are designed to have a 1 or near-1 entropy efficiency, and proofs of correctness and efficiency are contained throughout this section.

We’ll use uppercase letters X , Y and Z for discrete random variables, and write $H(X)$ for the Shannon entropy of this variable. Write $X \sim \text{Unif}\{0 \dots x - 1\}$ to mean that X has a discrete uniform distribution of size x , or $X \sim \text{Unif}\{n\}$ for short. $H(\text{Unif}\{n\}) = \log n$. $\text{Bern}\{p\}$ is a Bernoulli distribution with parameter p . $A \perp\!\!\!\perp B$ means that random variables A and B are independent. The interval $[a, b)$ means all integers in the range $a \dots b - 1$. Write $\mathbb{P}$ for probability, $\mathbb{R}$ for the set of real numbers, and lowercase f as a function on random variables. In algorithms, $\div$ and mod are integer division and modulus operations, and subscripts [...] are used for array access.

We’ll say “uniform variable” to mean a uniformly distributed integer variable or variate.

A. Basic operations

At the basis of the ES is the ability to transform a uniform variable into different forms. We can map a uniform variable into (a) two uniform variables, (b) a Bernoulli variable and a uniform variable, (c) a weighted variable and a uniform variable. These transformations preserve entropy. The contribution is how to combine these transformations for efficient entropy conversion.

The function f_{combine} , in Definition 1, allows us to combine two uniform variables X and Y into a single uniform variable Z . Its inverse f_{divide}

is given in Definition 2. $f_{combine}$ and f_{divide} map an interval of size xy into x intervals of size y as follows:

$$\underbrace{\overbrace{0 \dots y-1}^{Z \sim \text{Unif}\{xy\}} \underbrace{y \dots 2y-1 \dots y(x-1) \dots xy-1}_{Y \sim \text{Unif}\{y\} \quad Y \sim \text{Unif}\{y\}}}_{X \sim \text{Unif}\{x\}}$$

Definition 1. Let $f_{combine} : [0, x] \times [0, y] \rightarrow [0, xy]$ be $f_{combine} : (X, Y) \mapsto yX + Y$.

Definition 2. Let $f_{divide} : [0, xy] \rightarrow [0, x] \times [0, y]$ be $f_{divide} : Z \mapsto (\lfloor \frac{Z}{y} \rfloor, Z \bmod y)$.

Lemma 1. $f_{combine}$ is a bijection with inverse f_{divide} .

Proof. $f_{combine}(f_{divide}(Z)) = f_{combine}(\lfloor \frac{Z}{y} \rfloor, Z \bmod y) = y \lfloor \frac{Z}{y} \rfloor + (Z \bmod y) = Z$. $f_{divide}(f_{combine}(X, Y)) = f_{divide}(yX + Y) = (\lfloor \frac{yX+Y}{y} \rfloor, (yX + Y) \bmod y) = (\lfloor X + \frac{Y}{y} \rfloor, Y) = (X, Y)$. Therefore $f_{combine}$ is a bijection. $\square$

Lemma 2. If $X \sim \text{Unif}\{x\}$, $Y \sim \text{Unif}\{y\}$ and $X \perp\!\!\!\perp Y$, then $f_{combine}(X, Y) \sim \text{Unif}\{xy\}$.

Proof. Let $Z = yX + Y$, so $Z \in [0, xy]$. $\mathbb{P}(z = Z) = \mathbb{P}(z = yX + Y) = \sum_{i=0}^{x-1} \mathbb{P}(z = yX + Y | X = i) \mathbb{P}(X = i) = \frac{1}{m} \mathbb{P}(X = i) = \frac{1}{y} \frac{1}{x} = \frac{1}{xy}$. Therefore, Z is uniformly distributed over $[0, xy]$. $\square$

Lemma 3. If $Z \sim \text{Unif}\{xy\}$, and $f_{divide}(Z) = (X, Y)$, then $X \sim \text{Unif}\{x\}$ and $Y \sim \text{Unif}\{y\}$. $X \perp\!\!\!\perp Y$.

Proof. $\mathbb{P}(X = i) = \mathbb{P}(X = \lfloor \frac{Z}{y} \rfloor) = \frac{1}{x}$. $\mathbb{P}(Y = j) = \mathbb{P}(Y = Z \bmod y) = \frac{1}{y}$. $\square$

Corollary 1. $f_{combine}$ and f_{divide} do not lose entropy.

Proof. $f_{combine}$ and f_{divide} are bijections so must preserve entropy. $H(X, Y) = H(X) + H(Y) = \log n + \log m = \log(nm) = H(Z)$, so the entropy on both sides of the transformation is equal. $\square$

Algorithm 1 is an implementation of $f_{combine}$, and Algorithm 2 is an implementation of f_{divide} ,

with the postconditions established by Lemmas 2 and 3.

Algorithm 1 Combining two uniform variables into one uniform variable

Require: X, Y, x and y are integers

Require: $X \sim \text{Unif}\{x\}$

Require: $Y \sim \text{Unif}\{y\}$

Require: $X \perp\!\!\!\perp Y$

Ensure: z is $x * y$

Ensure: $Z \sim \text{Unif}\{z\}$

1: **procedure** COMBINE(X, x, Y, y)

2: $Z \leftarrow y * X + Y$

3: $z \leftarrow x * y$

4: **return** Z, z

5: **end procedure**

Algorithm 2 Converting a uniform variable into two uniform variables by division

Require: Z, z and y are integers

Require: $z \bmod y = 0$, i.e. z is divisible by y

Require: $Z \sim \text{Unif}\{z\}$

Ensure: $x * y = z$

Ensure: $X \sim \text{Unif}\{x\}$

Ensure: $Y \sim \text{Unif}\{y\}$

Ensure: $X \perp\!\!\!\perp Y$

1: **procedure** DIVIDE(Z, z, y)

2: $X \leftarrow Z \div y$

3: $Y \leftarrow Z \bmod y$

4: $x \leftarrow z \div y$

5: **return** X, x, Y

6: **end procedure**

The function f_{sample} , in Definition 3, maps the interval $[0, z]$ into the intervals $[0, x]$ and $[x, z]$ as follows:

$$\underbrace{\overbrace{0 \dots x-1}^{Z \sim \text{Unif}\{z\}} \underbrace{x \dots z-1}_{Y_0 \sim \text{Unif}\{x\} \quad Y_1 \sim \text{Unif}\{z-x\}}}_{X \sim \text{Bern}\{\frac{x}{z}\}}$$

Definition 3. Let $f_{sample} : Z \mapsto (Z < x, Z - x(Z \geq x))$. The operators $<$ and $\geq$ evaluate to 0 or 1.

Lemma 4. If $Z \sim \text{Unif}\{z\}$ and $f_{\text{sample}}(Z) = (X, Y)$, then $X \sim \text{Bern}\{\frac{x}{z}\}$ and $Y \sim \text{Unif}\{Xx + (1 - X)(z - x)\}$. $X \perp\!\!\!\perp Y$.

Proof. $\mathbb{P}(X = 0) = \mathbb{P}(Z < x) = \frac{x}{z}$, $\mathbb{P}(X = 1) = \mathbb{P}(Z \geq x) = \frac{z-x}{z} = 1 - \frac{x}{z} = 1 - \mathbb{P}(X = 0)$, therefore $X \sim \text{Bern}\{\frac{x}{z}\}$.

If $X = 0$, then $Z < x$ and $\mathbb{P}(Y = j) = \mathbb{P}(Z = j | Z < x) = \frac{1}{x}$, therefore $Y \sim \text{Unif}\{x\}$. If $X = 1$ then $Z \geq x$ and $\mathbb{P}(Y = j) = \mathbb{P}(Z = Y + j | Z \geq x) = \frac{1}{z-x}$, therefore $Y \sim \text{Unif}\{z - x\}$. In both cases $X \perp\!\!\!\perp Y$. $\square$

Algorithm 3 implements f_{sample} . Algorithm 3 can be used to resize a uniform variable or to generate a Bernoulli variable.

Algorithm 3 Converting a uniform variable into a Bernoulli and a uniform variable

Require: Z, x and z are integers

Require: $0 \leq x \leq z$

Require: $Z \sim \text{Unif}\{z\}$

Ensure: $X \sim \text{Bern}\{\frac{x}{z}\}$

Ensure: $Y \sim \text{Unif}\{y\}$

Ensure: $y = Xx + (1 - X)(z - x)$

Ensure: $X \perp\!\!\!\perp Y$

```

1: procedure SAMPLE( $Z, z, x$ )
2:   if  $Z < x$  then
3:      $X \leftarrow 1$ 
4:      $y \leftarrow x$ 
5:      $Y \leftarrow Z$ 
6:   else
7:      $X \leftarrow 0$ 
8:      $y \leftarrow z - x$ 
9:      $Y \leftarrow Z - x$ 
10:  end if
11:  return  $Y, y, X$ 
12: end procedure

```

Lemma 5. f_{sample} preserves entropy.

Proof. If $f_{\text{sample}}(Z) = (X, Y)$, then the entropy in $X \sim \text{Bern}\{\frac{x}{z}\}$ is given by the binary entropy function [4]:

$$H(X) = -\frac{x}{z} \log \frac{x}{z} - \frac{z-x}{z} \log \frac{z-x}{z} \quad (1)$$

The output entropy

$$H(f_{\text{sample}}(Z)) \quad (2)$$

$$= H(X, Y) \quad (3)$$

$$= H(X) + H(Y) \quad (4)$$

$$= H(X) + \mathbb{P}(X = 0)H(Y|X = 0) + \mathbb{P}(X = 1)H(Y|X = 1) \quad (5)$$

$$= -\frac{x}{z} \log \frac{x}{z} - \frac{z-x}{z} \log \frac{z-x}{z} + \frac{x}{z} \log x + \frac{z-x}{z} \log(z-x) \quad (6)$$

$$= -\frac{x}{z} \log x + \frac{x}{z} \log z - \frac{z-x}{z} \log(z-x) + \frac{z-x}{z} \log z + \frac{x}{z} \log x + \frac{z-x}{z} \log(z-x) \quad (7)$$

$$= \frac{x}{z} \log z + \frac{z-x}{z} \log z \quad (8)$$

$$= (\frac{x}{z} + \frac{z-x}{z}) \log z \quad (9)$$

$$= \log z \quad (10)$$

$$= H(Z) \quad (11)$$

which equals the input entropy, so therefore f_{sample} preserves entropy. $\square$

Corollary 2. Algorithms 1, 2 and 3 preserve entropy, from Corollary 1 and Lemma 5.

Algorithms 1 and 3 could be used to implement von Neumann's rejection sampling algorithm [23] and Lumbrusco's FDR algorithm [22]. The reason why rejection sampling and FDR lose entropy is because they discard one or both outputs from the SAMPLE algorithm.

B. Generating uniform variables

The first step is to obtain a uniform variable $S \sim \text{Unif}\{xy\}$ where x is the number you need to generate, and y is some integer. The key insight is that we can use SAMPLE to resize $S \sim \text{Unif}\{s\}$ very slightly to $S \sim \text{Unif}\{s - s \bmod x\}$ with almost no entropy loss, and the resulting size will be divisible x .

Algorithm 4 generates a uniform variable $S \sim \text{Unif}\{xy\}$, by combining binary or uniform entropy into an existing store (line 4) until it reaches a minimum size $s_{\min}$ (line 3). Then it resizes S to be a multiple of x on line 8, and if successful,

returns the result on line 10. If it fails ($B = 0$), then repeat. The reason this is efficient is because when x is much smaller than s_{min} , it is overwhelmingly likely that $B = 1$, and so the lost entropy in B (on line 8) is very small. Algorithm 4 terminates with probability 1. b is the base of `FETCH()` and is often equal to 2.

Algorithm 5 generates a uniform variable $X \sim \text{Unif}\{x\}$ using an entropy source and an entropy store $S \sim \text{Unif}\{s\}$. Unused entropy is returned in S . The algorithm calls `GENERATE_MULTIPLE` to create a uniform variable $S \sim \text{Unif}\{xy\}$, then calls `DIVIDE` (Algorithm 2) to split S into $S \sim \text{Unif}\{y\}$ and $X \sim \text{Unif}\{x\}$. X is our result, and S is the new entropy store. A C implementation of Algorithm 5 is given in the Appendix.

Algorithm 4 Generating a uniform multiple

Require: S, s, s_{min}, x and b are integers

Require: $x \leq s_{min}$

Require: $b \geq 2$

Require: `FETCH()` $\sim \text{Unif}\{b\}$

Require: $S \sim \text{Unif}\{s\}$

Ensure: $S \sim \text{Unif}\{xy\}$

Ensure: $s = xy$

```

1: procedure GENERATE_MULTIPLE( $S, s, x$ )
2:   while True do
3:     while  $s < s_{min}$  do
4:        $S, s \leftarrow \text{COMBINE}(S, s, \text{FETCH}(), b)$ 
5:     end while
6:      $y \leftarrow s \div x$ 
7:      $m \leftarrow s \bmod x$ 
8:      $S, s, B \leftarrow \text{SAMPLE}(S, s, s - m)$ 
9:     if  $B = 1$  then
10:      return  $S, s, y$ 
11:    end if
12:  end while
13: end procedure

```

Lemma 6. In Algorithm 5, $X \sim \text{Unif}\{x\}$, $S \sim \text{Unif}\{s\}$ and $S \perp\!\!\!\perp X$.

Proof. The preconditions of `DIVIDE` on line 3 are met, because $s = xy$ implies that s is divisible by x . Therefore the postconditions of `DIVIDE` are also met, so $X \sim \text{Unif}\{x\}$, $S \sim \text{Unif}\{s\}$ and $S \perp\!\!\!\perp X$.

Algorithm 5 Generating a uniform variable of a given size

Require: S, s and x are integers

Require: $S \sim \text{Unif}\{s\}$

Ensure: $X \sim \text{Unif}\{x\}$

Ensure: $S \sim \text{Unif}\{s\}$

Ensure: $S \perp\!\!\!\perp N$

```

1: procedure GENERATE_UNIFORM( $S, s, x$ )
2:    $S, s, y \leftarrow \text{GENERATE_MULTIPLE}(S, s, x)$ 
3:    $S, s, X \leftarrow \text{DIVIDE}(S, s, x)$ 
4:   return  $S, s, X$ 
5: end procedure

```

N . □

Lemma 7. For $p, q \in \mathbb{R}$, where $0 \leq p \leq q \leq \frac{1}{2}$,

$$-p \log p - (1-p) \log(1-p) \quad (12)$$

$$\leq -q \log q - (1-q) \log(1-q) \quad (13)$$

Proof. Let

$$g(p) = -p \log p - (1-p) \log(1-p) \quad (14)$$

$$\Rightarrow g'(p) = \log \frac{1-p}{p} \quad (15)$$

$$= \log\left(\frac{1}{p} - 1\right) \quad (16)$$

$$\geq \log\left(\frac{1}{\frac{1}{2}} - 1\right) \quad (17)$$

$$= \log 1 \quad (18)$$

$$= 0 \quad (19)$$

Since the derivative of $g \geq 0$ it means that g is monotonic in the range. □

Definition 4. Let ϵ be the expected entropy loss function of Algorithm 4. Let ϵ_{max} be the upper bound of ϵ .

Theorem 1. If $p = \frac{x-1}{s_{min}} \leq \frac{1}{2}$, then

$$0 \leq \epsilon \leq -\frac{p}{1-p} \log p - \log(1-p) \quad (20)$$

Proof. For each iteration i of Algorithm 4, let $p_i = \frac{s_i \bmod x}{s_i}$. But $(s_i \bmod x) \leq (x-1)$ and $s_i \geq s_{min}$, so $\frac{s_i \bmod x}{s_i} \leq \frac{x-1}{s_{min}}$, so $p_i \leq p$. On each

iteration, the entropy lost is equal to the entropy in the variable $B_i \sim \text{Bern}\{p_i\}$, which is given by the entropy equation for a Bernoulli distribution [4]:

$$H(B_i) = -p_i \log p_i - (1 - p_i) \log(1 - p_i) \quad (21)$$

Therefore,

$$H(B_i) \leq -p \log p - (1 - p) \log(1 - p) \quad (22)$$

by Lemma 7. The expected number of iterations m is given by

$$m = 1 + p_i m \leq 1 + pm \quad (23)$$

$$\implies m - pm \leq 1 \quad (24)$$

$$\implies m(1 - p) \leq 1 \quad (25)$$

$$\implies m \leq \frac{1}{1 - p} \quad (26)$$

The total entropy ϵ lost by the algorithm is given by the number of iterations m of the algorithm multiplied by the entropy $H(B_i)$ lost in each iteration.

$$\epsilon = mH(B_i) \quad (27)$$

$$\leq \frac{1}{1 - p} (-p \log p - (1 - p) \log(1 - p)) \quad (28)$$

$$= -\frac{p}{1 - p} \log p - \log(1 - p) \quad (29)$$

We can also end up in the situation where $(s \bmod n) = 0$ already, so SAMPLE succeeds with no entropy loss, so $\epsilon = 0$. $\square$

Corollary 3. If $p = \frac{x-1}{s_{min}} \leq \frac{1}{2}$, then the maximum entropy lost by Algorithm 4 is

$$\epsilon_{max} = -\frac{p}{1 - p} \log p - \log(1 - p) \quad (30)$$

Corollary 4. The maximum entropy loss of Algorithm 5 is ϵ_{max} .

Proof. Algorithm 5 loses up to ϵ_{max} entropy in its call to GENERATE_MULTIPLE on line 2, and the DIVIDE on line 3 does not lose entropy according to Corollary 1. $\square$

Corollary 5. If $p = \frac{x-1}{s_{min}} \leq \frac{1}{2}$, then the minimum efficiency η_{min} of Algorithm 5 is

$$\eta_{min} = \frac{\log x}{\log x + \epsilon_{max}} \quad (31)$$

Proof.

$$\eta = \frac{H_{out}}{H_{in}} = \frac{H_{out}}{H_{out} + \epsilon_{max}} = \frac{\log x}{\log x + \epsilon_{max}} \quad (32)$$

$\square$

To illustrate Equation 31, if $s_{min} = 2^{31}$ and $x = 6$, then $\eta \geq 0.99999997$. This means that even with a modest entropy buffer, we can get very good entropy efficiency.

Theorem 2. The efficiency of an entropy store of n bits is

$$\eta_{min} = 1 - \Theta(n2^{-n}) \quad (33)$$

Proof. This follows from (31), where we keep x constant. The number of bits n required by the ES is $\log s_{max}$, where s_{max} is the maximum value in the store. The maximum value is b times the minimum value s_{min} (see Algorithm 4), therefore

$$n = \log(s_{max}) = \log(bs_{min}) \quad (34)$$

$$\implies 2^n = bs_{min} \quad (35)$$

$$\implies s_{min} = \frac{2^n}{b} \quad (36)$$

From the definition of p ,

$$p = \frac{x - 1}{s_{min}} \quad (37)$$

$$= \frac{b(x - 1)}{2^n} \quad (38)$$

$$= 2^{-n}b(x - 1) \quad (39)$$

$$= \Theta(2^{-n}) \quad (40)$$

$$-\log p = -\log(2^{-n}b(x - 1)) \quad (41)$$

$$= -\log(2^{-n}) - \log(b(x - 1)) \quad (42)$$

$$= n - \log(b(x - 1)) \quad (43)$$

$$= \Theta(n) \quad (44)$$

We also have

$$1 - p = \Theta(1) \quad (45)$$

An expansion of $\ln(1 + p)$ yields

$$\frac{p}{1 + p} \leq \ln(1 + p) \leq p \quad (46)$$

from which it follows that

$$-\log(1 - p) = \Theta(p) = \Theta(2^{-n}) \quad (47)$$

From our definition of ϵ_{max} (30), and substituting in (40), (44), (45) and (47),

$$\epsilon_{max} = -\frac{p}{1 - p} \log p - \log(1 - p) \quad (48)$$

$$= \frac{\Theta(2^{-n})}{\Theta(1)} \Theta(n) + \Theta(2^{-n}) \quad (49)$$

$$= \Theta(n2^{-n}) + \Theta(2^{-n}) \quad (50)$$

$$= \Theta(n2^{-n}) \quad (51)$$

The efficiency, defined in (31), and substituting in (51), is

$$\eta_{min} = \frac{H_{out}}{H_{out} + \epsilon_{max}} \quad (52)$$

$$= 1 - \frac{\epsilon_{max}}{H_{out} + \epsilon_{max}} \quad (53)$$

$$= 1 - \frac{\Theta(n2^{-n})}{\Theta(1)} \quad (54)$$

$$= 1 - \Theta(n2^{-n}) \quad (55)$$

□

Corollary 6. As $s_{min} \rightarrow \infty$, $\eta_{min} \rightarrow 1$.

Corollary 7. By increasing s_{min} , the entropy efficiency η of Algorithm 5 can be arbitrarily close to 1.

C. Generating Bernoulli variables

Algorithm 6 generates a Bernoulli variable $B \sim \text{Unif}\{\frac{y}{x}\}$ using an entropy source and an entropy store $S \sim \text{Unif}\{s\}$. Unused entropy is returned in S . The call to GENERATE_MULTIPLE ensures that the store $S \sim \text{Unif}\{mx\}$ so we can apply SAMPLE

(Algorithm 3) to extract $S \sim \text{Unif}\{my\}$ or $S \sim \text{Unif}\{mx - my\}$ from S . We return the Bernoulli output B and recycle the remaining S as the new entropy store.

Algorithm 6 Generating a Bernoulli variable

Require: S, s, y, x are integers

Require: $S \sim \text{Unif}\{s\}$

Require: $0 \leq y \leq x \leq s_{min}$

Ensure: $B \sim \text{Bern}\{\frac{y}{x}\}$

Ensure: $S \sim \text{Unif}\{s\}$

Ensure: $S \perp\!\!\!\perp B$

```

1: procedure GENERATE_BERNOULLI( $S, s, y, x$ )
2:    $S, s, m \leftarrow \text{GENERATE\_MULTIPLE}(S, s, x)$ 
3:    $S, s, B \leftarrow \text{SAMPLE}(S, s, m * y)$ 
4:   return  $S, s, B$ 
5: end procedure

```

Lemma 8. Algorithm 6 returns $S \sim \text{Unif}\{s\}$ and $B \sim \text{Bern}\{\frac{y}{x}\}$. $B \perp\!\!\!\perp S$.

Proof. GENERATE_MULTIPLE creates $S \sim \text{Unif}\{my\}$ on line 2, then SAMPLE returns $B \sim \text{Bern}\{\frac{my}{mx}\} = \text{Bern}\{\frac{y}{x}\}$, and $S \sim \text{Unif}\{s\}$. By Lemma 4, $B \perp\!\!\!\perp S$. □

Lemma 9. The entropy loss of Algorithm 6 is given by $\epsilon(\frac{x-1}{s_{min}})$.

Proof. The only entropy loss from Algorithm 6 is the call to GENERATE_MULTIPLE on line 2, which loses $\epsilon(\frac{x-1}{s_{min}})$ according to Theorem 1. □

Corollary 8. The efficiency of Algorithm 6 is $1 - \Theta(n2^{-n})$.

□

D. Generating weighted integer variables

Algorithm 7 generates a weighted variable $W \sim \text{Weighted}\{w_0, \dots, w_{k-1}\}$ using an entropy source and an entropy store $S \sim \text{Unif}\{s\}$. Unused entropy is returned in S . The distribution has k outcomes where each outcome has integer weight $\{w_0, w_1, \dots, w_{k-1}\}$, giving output probabilities $\{\frac{w_0}{x}, \frac{w_1}{x}, \dots, \frac{w_{k-1}}{x}\}$, where $x = \sum_i w_i$ is the total weight.

The algorithm works by mapping the x outcomes of a uniform distribution to the k outputs of the

weighted distribution using a lookup table. An implementation of this could be an array of size x (construction time $O(x)$, lookup time $O(1)$), or a binary search on an array of size k (construction time $O(k)$, lookup time $O(\log k)$).

The interval $[0, x)$ maps to k intervals with size w_i as follows:

$$\begin{array}{c} \overbrace{0 \dots w_0 - 1 \quad w_0 \dots w_0 + w_1 - 1 \quad \dots \quad x - w_{k-1} \dots x - 1}^{Z \sim \text{Unif}\{x\}} \\ \underbrace{Y_0 \sim \text{Unif}\{w_0\} \quad Y_1 \sim \text{Unif}\{w_1\} \quad \dots \quad Y_{k-1} \sim \text{Unif}\{w_{k-1}\}}_{X \sim \text{Weighted}\{w_0, \dots, w_{k-1}\}} \end{array}$$

When we map Z to the output variable X , then we can also produce a uniform variable $Y \sim \text{Unif}\{w_i\}$ which contains $\log w_i$ entropy, which we can recycle into the entropy store. Lemma 11 shows us that when we account for this extra entropy then no entropy is lost.

Definition 5. For integer weights $w_i \geq 0$, $0 \leq i < k$, let the offset $o_0 = 0$, $o_{i+1} = o_i + w_i$.

Definition 6. Let the output $t_j = i$ for $o_i \leq j < o_{i+1}$.

Definition 7. Define $f_{\text{weighted}} : Z \mapsto (t_Z, Z - o_{t_Z})$.

Lemma 10. If $Z \sim \text{Unif}\{x\}$ and $f_{\text{weighted}} : Z \mapsto (X, Y)$, then $X \sim \text{Weighted}\{w_0, \dots\}$ and $Y \sim \text{Unif}\{w_X\}$.

Proof. $\mathbb{P}(X = i) = \mathbb{P}(X = t_Z) = \frac{o_{i+1} - o_i}{x} = \frac{w_i}{x}$. Therefore $X \sim \text{Weighted}\{w_0, \dots\}$. $\mathbb{P}(Y = y) = \mathbb{P}(Y = Z - o_X | X = i) = \mathbb{P}(Y = Z - o_i | o_i \leq Z < o_{i+1}) = \frac{1}{o_{i+1} - o_i} = \frac{1}{w_i}$. Therefore $Y \sim \text{Unif}\{w_X\}$ and $X \perp\!\!\!\perp Y$. $\square$

f_{weighted} is a generalisation of f_{combine} and f_{sample} . Although X is independent of Y , the size of Y is not independent of X .

Lemma 11. f_{weighted} does not lose entropy.

Proof. Let $q_i = \frac{w_i}{x} = \mathbb{P}(X = i)$.

$$\begin{aligned} & H(f_{\text{weighted}}(Z)) \\ &= H(X, Y) \end{aligned} \tag{56}$$

$$= H(X) + H(Y) \tag{57}$$

$$\begin{aligned} &= - \sum_i q_i \log q_i + \\ &\quad \sum_i \mathbb{P}(X = i) H(Y | X = i) \end{aligned} \tag{58}$$

$$= - \sum_i q_i \log \frac{w_i}{x} + \sum_i q_i \log w_i \tag{59}$$

$$\begin{aligned} &= - \sum_i q_i (\log w_i - \log x) + \\ &\quad \sum_i q_i \log w_i \end{aligned} \tag{60}$$

$$\begin{aligned} &= - \sum_i q_i \log w_i + \sum_i q_i \log x + \\ &\quad \sum_i q_i \log w_i \end{aligned} \tag{61}$$

$$= \sum_i q_i \log x \tag{62}$$

$$= (\log x) \sum_i q_i \tag{63}$$

$$= \log x \tag{64}$$

$$= H(Z) \tag{65}$$

Therefore the entropy of the input and outputs of f_{weighted} is the same, and f_{weighted} does not lose any entropy. $\square$

Algorithm 7 generates a weighted variable by first calling `GENERATE_MULTIPLE` to generate $S \sim \text{Unif}\{mx\}$. Lines 3-5 implement f_{weighted} to yield the weighted variable W and a uniform variable $S \sim \text{Unif}\{mw_W\}$. The weights have been scaled by a factor of m to avoid recomputing the lookup tables.

Lemma 12. The entropy loss of Algorithm 7 is given by $\epsilon(\frac{x-1}{s_{\min}})$.

Proof. By Lemma 11, the only entropy-losing step in Algorithm 7 is the call to `GENERATE_MULTIPLE`, which loses $\epsilon(\frac{x-1}{s_{\min}})$ entropy by Theorem 1. $\square$

Algorithm 7 Generating a weighted variable**Require:** S, s and x are integers**Require:** An array w of k integer weights $w_i \geq 0$ totalling x **Require:** An array t of x integer outputs (see Definition 6)**Require:** An array o of k integer offsets (see Definition 5)**Require:** $S \sim \text{Unif}\{s\}$ **Ensure:** $S \sim \text{Unif}\{s\}$ **Ensure:** $W \sim \text{Weighted}\{w_0, \dots\}$ **Ensure:** $S \perp\!\!\!\perp W$

```

1: procedure GEN_WEIGHTED( $S, s, x, w, t, o$ )
2:    $S, s, m \leftarrow \text{GENERATE\_MULTIPLE}(S, s, x)$ 
3:    $W \leftarrow t_{[S \div m]}$ 
4:    $s \leftarrow m * w_{[W]}$ 
5:    $S \leftarrow S - m * o_{[W]}$ 
6:   return  $S, s, W$ 
7: end procedure

```

III. EVALUATION

We can evaluate a random number generator based on its speed, memory footprint, setup time, entropy efficiency and quality of output distribution. [28]

For entropy efficiency, we can compare entropy store against entropy-optimal algorithms Knuth-Yao (KY) and the Interval Algorithm (IA).

Figure 1 shows the calculated entropy loss for generating uniform variables, comparing ES with von Neumann (vN) and KY generators. As expected (see for example [2] for a detailed analysis), the entropy loss of vN and KY generators depend on the number generated, with KY losing up to 2 bits per output.

This figure shows the entropy loss calculated using Equation 30 for a 32-bit and a 64-bit entropy buffer, with an increase in buffer size yielding a higher efficiency. We also show sample real-world entropy losses confirming that ϵ_{max} is an upper bound.

Figure 2 shows the overall impact of entropy losses when applied to the shuffling a deck of N cards using the Fisher-Yates algorithm [8, 10, 18]. This is a good demonstration of the entropy

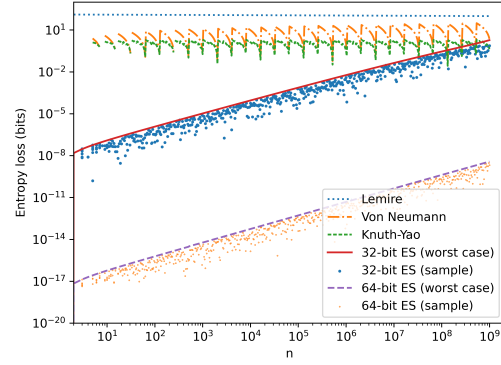


Fig. 1. Calculated entropy losses generating $\text{Unif}\{n\}$ for different algorithms.

efficiency under a real-world workload. The total entropy loss of ES is $\sum_{n=2}^N \epsilon(\frac{n-1}{s_{min}})$, so with a 32-bit buffer, shuffling a deck of 52 cards can be done with an entropy loss of $\leq 1.7 \times 10^{-5}$ bits, or an entropy efficiency of ≥ 0.99999992 .

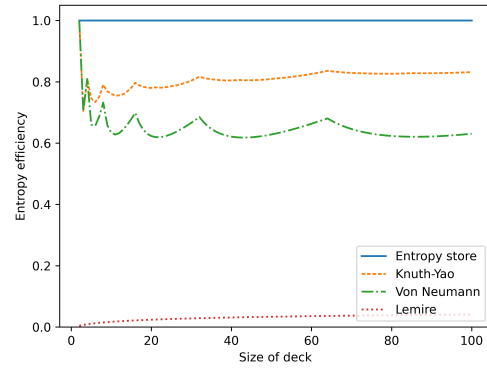


Fig. 2. Calculated entropy efficiency shuffling cards using the Fisher-Yates algorithm, using different uniform generator algorithms.

When generating Bernoulli variables, we see in Figure 3 that the entropy efficiency of IA drops off significantly at lower entropy. IA has spikes at the dyadic points that are more efficient to generate. IA must fetch between 1 and 2 bits per output on average. By contrast, ES algorithms do not

necessarily fetch any bits to generate an output, as there may be enough entropy in the store already, so ES just shrinks the size of its store to generate an output. Figure 4 shows that IA cannot significantly increase its output rate for low-entropy outputs.

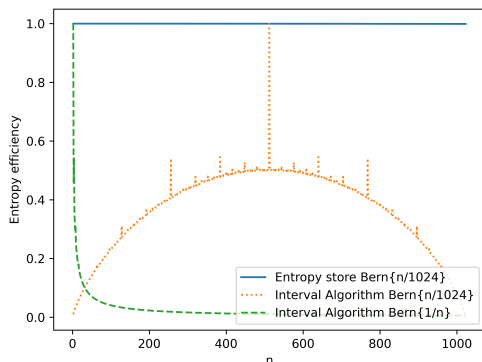


Fig. 3. Calculated entropy efficiency generating Bernoulli variables for different algorithms.

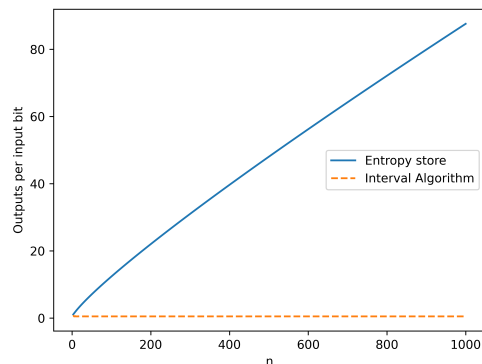


Fig. 4. Calculated output rate for Bernoulli distribution $\text{Bern}\{\frac{1}{n}\}$ for different algorithms.

For practical purposes, Algorithms 5, 6 and 7 are $O(1)$. The setup for Algorithms 5 and 6 are $O(1)$, but Algorithm 7 requires $O(k)$ setup time and space. Integer division and multiplication is actually $O(n \log n)$ [14], so for an n bit ES the time per output is $O(n \log n)$.

From a theoretical perspective, the efficiency increases as $1 - \Theta(n2^{-n})$, which converges to 1 much more quickly than batching algorithms at $1 - \Theta(n^{-1})$, which is why the entropy store can fit in a 32-bit integer.

Table II compares the speed of ES against various best in class random number generators. These numbers are heavily compiler and CPU dependent, so are only a guide.

When reading entropy from a hardware random generator, rate of entropy input is the dominating factor and the ES is the fastest by a significant margin. When using a very fast pseudo-random source like Xorisho-128 [3], we see that Lemire's algorithm [21] dominates, as this is designed for speed over entropy efficiency.

Speed is best understood in terms of CPU pipeline architecture. Branching, memory access, hardware random numbers and integer divmod operations are all relatively expensive operations [1]. Table-based methods like FLDR [30] and ALDR [28] perform memory access which can be slower than purely arithmetic methods. ES is likely to suffer due to its 2 integer divmod operations, but compilers can strength-reduce integer division to a multiplication and a shift [12], which is shown in the "optimized" version of these algorithms in Table II. ES was the fastest for Bernoulli outputs in all cases because there is only one divmod operation needed per output.

Overall, ES is most suited when generating true random variables from a hardware entropy source. When a pseudo-random source is used, then Lemire's algorithm is the fastest. Although a 64-bit ES is more efficient than a 32-bit ES, the slower 64-bit operations outweigh the marginal improvement in efficiency.

IV. CONCLUSION AND FURTHER WORK

We have introduced a new class of algorithm to generate random variables using a uniform entropy store, and shown that by recycling unused entropy between outputs we can achieve arbitrarily low entropy losses. We have shown that the entropy efficiency limits of classical algorithms like Knuth-Yao [19] and the Interval Algorithm [13] can be

Algorithm	Output	H/W	Xoshiro
ES32	Unif{6}	1	1
ES32 optimized		1	0.47
ES64		1	1.24
ES64 optimized		1	0.55
vN [23]		1.55	0.77
FDR [22]		1.42	0.83
Huber-Vergas [15]		1.42	1.0
FLDR [30]		1.42	1.46
ALDR [28]		1.42	1.5
Lemire [21]		24.8	0.26
ES32	Bern{ $\frac{1}{100}$ }	1	1
ES32 optimized		0.9	0.54
FLDR		28.3	3.8
ALDR		22.4	2.94
ES32	Weighted {1,2,3,4,5}	1	1
FLDR		1.36	1.38
ALDR		1.36	1.13

TABLE II

RELATIVE TIMES GENERATING VARIABLES FROM HARDWARE ENTROPY (H/W) AND XOSHIRO-128 [3] (LOWER IS BETTER).

overcome by using an entropy store, and that the efficiency converges to 1 faster than with traditional batching.

A big benefit of ES is that no setup step is needed for uniform and Bernoulli outputs, and that the same ES can be shared between different output types. ES is fastest when generating outputs from a hardware entropy source and for Bernoulli outputs, but is still competitive on pseudorandom entropy sources, particularly if the division operations can be optimized.

Further work could include improving the speed, generating other distributions, and determining if the current algorithms are optimal or if further efficiency improvements are possible for a given store size.

V. ADDENDUM

This paper was almost complete by the time I was completing the final literature review, when I came across this preprint by Draper and Saad [6], published on ArXiv just a few months earlier. The similarities with this work are striking, and unfortunately because it was very new I had not come across it before. Of course I have followed Saad and Draper’s work with interest, and none of their work up to that point had come anywhere near what I was working on. But now most equations, diagrams and

algorithms are old news. Whilst validating, it’s also a shame as of course as researchers we try to avoid treading on each other’s toes.

A major point of difference is whether we need to count the buffered entropy against our own efficiency, as indicated by the $O(1/k)$ term in the entropy consumption. I would argue that as there are no real limits on the batch size - it is an online algorithm after all - and we can just assume that the entropy buffer is already full and count it as part of the input, and we only need to measure our delta against the entropy buffer.

Of course I wish Draper and Saad the very best, they had no reason to suspect I was already working on this as well, it is just one of those things that happens.

Further implementation and notes can be found at <https://github.com/calum74/entropy>
- Calum Grant, 17th November 2025

REFERENCES

- [1] Andreas Abel and Jan Reineke. “uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures”. In: *ASPLOS*. ASPLOS ’19. Providence, RI, USA: ACM, 2019, pp. 673–686. ISBN: 978-1-4503-6240-5. DOI: 10.1145/3297858.3304062. URL: <http://doi.acm.org/10.1145/3297858.3304062>.
- [2] Axel Bacher et al. “Generating Random Permutations by Coin Tossing: Classical Algorithms, New Analysis, and Modern Implementation”. In: *ACM Trans. Algorithms* 13.2 (Feb. 2017). ISSN: 1549-6325. DOI: 10.1145/3009909. URL: <https://doi.org/10.1145/3009909>.
- [3] David Blackman and Sebastiano Vigna. “Scrambled Linear Pseudorandom Number Generators”. In: *ACM Trans. Math. Softw.* 47.4 (Sept. 2021). ISSN: 0098-3500. DOI: 10.1145/3460772. URL: <https://doi.org/10.1145/3460772>.
- [4] Thomas M Cover. *Elements of information theory*. John Wiley & Sons, 1999.
- [5] Luc Devroye. *Non-Uniform Random Variate Generation*. Springer-Verlag, 1986.

- [6] Thomas L Draper and Feras A Saad. “Efficient Online Random Sampling via Randomness Recycling”. In: *arXiv preprint arXiv:2505.18879* (2025).
- [7] Thomas L Draper and Feras A Saad. “Efficient Rejection Sampling in the Entropy-Optimal Range”. In: *arXiv preprint arXiv:2504.04267* (2025).
- [8] Richard Durstenfeld. “Algorithm 235: random permutation”. In: *Communications of the ACM* 7.7 (1964), p. 420.
- [9] James Allen Fill and Mark Huber. “The randomness recycler: A new technique for perfect sampling”. In: *Proceedings 41st annual symposium on foundations of computer science*. IEEE, 2000, pp. 503–511.
- [10] Ronald Aylmer Fisher and Frank Yates. *Statistical tables for biological, agricultural and medical research*. Hafner Publishing Company, 1953.
- [11] Oded Goldreich. *Foundations of Cryptography, Volume 2*. Cambridge university press Cambridge, 2004.
- [12] Torbjörn Granlund and Peter L. Montgomery. “Division by invariant integers using multiplication”. In: *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*. PLDI ’94. Orlando, Florida, USA: Association for Computing Machinery, 1994, pp. 61–72. ISBN: 089791662X. DOI: 10.1145/178243.178249. URL: <https://doi.org/10.1145/178243.178249>.
- [13] Te Sun Han and Mamoru Hoshi. “Interval algorithm for random number generation”. In: *IEEE Transactions on Information Theory* 43 (Mar. 1997), pp. 599–611.
- [14] David Harvey and Joris Van Der Hoeven. “Integer multiplication in time $O(n \log n)$ ”. In: *Annals of Mathematics* 193.2 (2021), pp. 563–617.
- [15] Mark Huber and Danny Vargas. *Optimal rolling of fair dice using fair coins*. 2024. arXiv: 2412.20700 [cs.DS]. URL: <https://arxiv.org/abs/2412.20700>.
- [16] Mark L Huber. “Perfect simulation”. In: *Monographs on Statistics and Applied Probability* 148 (2016), p. 148.
- [17] David A. Huffman. “A Method for the Construction of Minimum-Redundancy Codes”. In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101. DOI: 10.1109/JRPROC.1952.273898.
- [18] Donald E Knuth. *The Art of Computer Programming: Seminumerical Algorithms, Volume 2*. Addison-Wesley Professional, 2014.
- [19] Donald Ervin Knuth and Andrew Chi-Chih Yao. “The complexity of nonuniform random number generation”. In: *Algorithms and Complexity: New directions and recent results*. Academic Press, New York, 1976, pp. 357–428.
- [20] Dexter Kozen and Matvey Soloviev. “Coalgebraic tools for randomness-conserving protocols”. In: *Journal of Logical and Algebraic Methods in Programming* 125 (2022), p. 100734. ISSN: 2352-2208. DOI: <https://doi.org/10.1016/j.jlamp.2021.100734>. URL: <https://www.sciencedirect.com/science/article/pii/S2352220821000973>.
- [21] Daniel Lemire. “Fast Random Integer Generation in an Interval”. In: *ACM Transactions on Modeling and Computer Simulation (TOMACS)* 29.1 (Jan. 2019). ISSN: 1049-3301. DOI: 10.1145/3230636. URL: <https://doi.org/10.1145/3230636>.
- [22] Jérémie Lumbroso. “Optimal discrete uniform generation from coin flips, and applications”. In: *arXiv preprint arXiv:1304.1916* (2013).
- [23] John von Neumann. “Various techniques used in connection with random digits”. In: *Notes by G.E. Forsythe, National Bureau of Standards, Applied Math Series* 12 (1951), pp. 36–38.
- [24] Yasutada Oohama. “Performance Analysis of the Interval Algorithm for Random Number Generation Based on Number Systems”. In: *IEEE Transactions on Information Theory* 57.3 (2011), pp. 1177–1185. DOI: 10.1109/TIT.2010.2103730.

- [25] Yasutada Oohama. “Performance analysis of the interval algorithm for random number generation in the case of Markov coin tossing”. In: *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* 103.12 (2020), pp. 1325–1336.
- [26] Sung-il Pae. “A Generalization of Peres’s Algorithm for Generating Random Bits From Loaded Dice”. In: *IEEE Transactions on Information Theory* 61.2 (2015), pp. 751–757. DOI: 10.1109/TIT.2014.2381223.
- [27] Yuval Peres. “Iterating von Neumann’s procedure for extracting random bits”. In: *The Annals of Statistics* (1992), pp. 590–597.
- [28] Feras A. Saad and Wonyeol Lee. “Random Variate Generation with Formal Guarantees”. In: *Proc. ACM Program. Lang.* 9.PLDI (June 2025). DOI: 10.1145/3729251. URL: <https://doi.org/10.1145/3729251>.
- [29] Feras A. Saad et al. “Optimal approximate sampling from discrete probability distributions”. In: *Proc. ACM Program. Lang.* 4.POPL (Dec. 2019). DOI: 10.1145/3371104. URL: <https://doi.org/10.1145/3371104>.
- [30] Feras A. Saad et al. “The Fast Loaded Dice Roller: A Near-optimal Exact Sampler for Discrete Probability Distributions”. In: *AISTATS 2020: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*. Vol. 108. Proceedings of Machine Learning Research. Palermo, Sicily, Italy: PMLR, 2020.
- [31] Claude E Shannon. “A mathematical theory of communication”. In: *The Bell system technical journal* 27.3 (1948), pp. 379–423.
- [32] Luca Trevisan et al. “Extractors and pseudo-random generators”. In: *Journal of the ACM* 48.4 (2001), pp. 860–879.
- [33] S. Vembu and S. Verdu. “Generating random bits from an arbitrary source: fundamental limits”. In: *IEEE Transactions on Information Theory* 41.5 (1995), pp. 1322–1332. DOI: 10.1109/18.412679.
- [34] M.D. Vose. “A linear algorithm for generating random numbers with a given distribution”. In: *IEEE Transactions on Software Engineering* 17.9 (1991), pp. 972–975. DOI: 10.1109/32.92917.
- [35] Alastair J Walker. “An efficient method for generating discrete random variables with general distributions”. In: *ACM Transactions on Mathematical Software (TOMS)* 3.3 (1977), pp. 253–256.
- [36] Shun Watanabe and Te Sun Han. “Interval Algorithm for Random Number Generation: Information Spectrum Approach”. In: *IEEE Transactions on Information Theory* 66.3 (2020), pp. 1691–1701. DOI: 10.1109/TIT.2019.2946235.

APPENDIX

Source code for GENERATE_UNIFORM, written in C.

```

const uint32_t s_min = 1<<31;
uint32_t s_value = 0;
uint32_t s_range = 1;

uint32_t
generate_uniform32(uint32_t n)
{
    for(;;)
    {
        // Preload entropy one bit
        // at a time into s
        while(s_range < s_min)
        {
            s_value <= 1;
            s_value |= fetch();
            s_range <= 1;
        }
        // Sample entropy s to a
        // multiple of n
        uint32_t r = s_range / n;
        uint32_t m = s_range % n;
        if(s_value >= m)
        {
            // Sample successful
            s_value -= m;
            uint32_t a = s_value / n;
            uint32_t b = s_value % n;

```

```
        s_value = a;  
        s_range = r;  
        return b;  
    }  
    else  
    {  
        // Reject  
        s_range = m;  
    }  
}
```